An Explainable Machine Learning Approach to Detecting Security Vulnerabilities in Python Code: A Comparative Evaluation Against Static Analysis

M.C. Roshan Perera (Student No. S25026203)
MSc Computer Science, Wrexham University

I. INTRODUCTION

A. Topic of Study

This proposal describes a dissertation project on vulnerability detection in Python source code. The project will build and test an offline tool using machine learning to detect insecure code. The tool will also describe each warning in plain English.

Python is one of the most popular programming languages. Used by developers for web services, data work, automation. Insecure Python code can therefore reach many systems. Common problems include SQL injection, command injection, hard-coded passwords and unsafe deserialisation. Several of these belong to the well-known OWASP list of web application risks [1]. Studies of Python packages show that such weaknesses appear in real projects and can stay hidden for a long time [2].

Most Python teams check security with static analysis. Bandit is a popular open-source example [3]. It checks code against fixed rules and reports patterns such as the use of eval or weak hash functions. Rule-based tools are fast and easy to use. However, they raise many false alarms, and developers often stop trusting them [4].

Machine learning offers another route. A model can learn patterns from labelled code instead of relying on hand-written rules. The bulk of prior work targets C and C++ code [5]. Python has received less attention.

This project will compare an explainable machine learning model with Bandit on Python code. It will also test a hybrid method that combines both. The main question of research is:

> *To what extent can an explainable machine learning model detect security vulnerabilities in Python source code more accurately than a rule-based static analyser (Bandit), and can a hybrid of the two reduce false positives?*

The project is product based. The product will be a Python application that runs without internet access. The dissertation will report on the research, design, implementation and evaluation of the product. The research will be based on publicly available datasets, allowing the project to generate its own results within the given timeframe.

B. Rationale

There are four reasons why this topic matters.

First, the problem is real. Vulnerabilities cost far more to fix after release than during development. A tool that points to the exact risky line, while the developer is still writing, saves time and money.

Second, static analysers have a known weakness. Johnson et al. found that false positives and unclear messages stop developers from using these tools [4]. Ayewah et al. describe how much effort teams spend deciding which warnings matter [6]. A tool that ranks warnings better would reduce this effort.

Third, research on machine learning for vulnerability detection has grown fast, but it has gaps. Chakraborty et al. showed that models which look strong on benchmark data lose a lot of accuracy on realistic data [5]. Many studies also say little about why a piece of code was flagged. A developer cannot act on a score without a reason. The need for checks is rising, because AI coding assistants can produce insecure code. Pearce et al. found that around 40 per cent of the programs generated by one assistant in their tests were vulnerable [7].

Fourth, the topic suits the scope of an MSc project. It uses free tools and public data. It needs no human participants. The final product can run from a disc without internet access, as the hand-over requires.

The expected contribution is a clear comparison on Python code, a tested hybrid method and an explanation layer that a developer can read. The results may also help small teams that cannot afford commercial scanners.

C. Requested Supervisor

For this project, I would prefer a supervisor who has experience in machine learning and cybersecurity. Some knowledge of software testing and experimental design would also be useful. I would also appreciate guidance on research methods and statistics, as these are areas in which I need to develop my knowledge and skills.

II. REVIEW OF RELEVANT LITERATURE

A. Background to Subject of Study

A.1 Software vulnerabilities in Python

Software vulnerability is a weakness that an attacker can use to harm a system. In Python, many weaknesses come from unsafe use of language features. Passing user input to eval or os.system can run attacker commands. Building SQL text by joining strings can allow injections. Loading data with pickle can run hidden code. Alfadel et al. studied vulnerabilities reported in Python packages. They found that the number of reports has grown and that many flaws stay in a package for years before discovery [2].

This project will focus on four classes of weakness. They are SQL injection (CWE-89), OS command injection (CWE-78), hard-coded credentials (CWE-798) and unsafe deserialisation (CWE-502). These four are common, easy to define and covered by Bandit rules. This makes a fair comparison possible.

A.2 Static analysis and Bandit

Static analysis checks code without running it. Bandit parses each Python file into an abstract syntax tree and runs a set of plugins over it [3]. Each plugin looks for one risky

pattern, for example a call to subprocess with shell=True. Each finding has a severity and a confidence level. Bandit is quick, free and works offline. These points make it a fair baseline for this project. Its rules are fixed, so it cannot judge context. It may flag harmless code in the same way as risky code.

A.3 Machine learning for vulnerability detection

Machine learning methods learn from examples of vulnerable and safe code. Early work used hand-made features, such as counts of risky function calls. Random forests suit this kind of data, because they handle mixed features and small datasets well [8]. Later work moved to deep learning on token sequences and code graphs, for example VulDeePecker [9] and Devign [10]. Pre-trained models such as CodeBERT turn a code snippet into a numeric vector [11]. Hanif et al. give a wide survey of these methods [12].

A.4 Explainable machine learning

A model score alone does not tell a developer what to fix. Explanation methods show which inputs drove a prediction. SHAP gives each feature a contribution value, based on ideas from game theory [13]. LIME explains a single prediction by fitting a simple model around it [14]. For tree models, SHAP values can be computed quickly. This makes SHAP practical for a tool that must run on an ordinary laptop.

B. Examples and Critique of Current Research in the Field

B.1 Static analysis tools

Johnson et al. interviewed developers and found that false positives and poorly presented warnings kept them from using static tools [4]. The strength of this study is that it reflects what developers say. Its limit is that it does not cover Python or Bandit. Ayewah et al. describe the effort of sorting through warnings at scale [6]. Both papers suggest that fewer and better ranked warnings would help. Neither tests whether machine learning could provide that ranking.

B.2 Machine learning approaches

VulDeePecker reported high detection rates using code gadgets and a neural network [9]. Devign used graph neural networks and reported gains over earlier models [10]. These results look strong. However, both works focus on C and C++. Their methods depend on tools that parse C code, so they do not transfer easily to Python.

Chakraborty et al. tested several deep learning models on a more realistic dataset. Performance fell sharply compared with results on older benchmarks [5]. They also pointed to problems such as duplicate samples and unrealistic class balance. This is a useful warning. In my project I must keep duplicates out of the test set and use a realistic share of safe code.

B.3 Datasets

CVEfixes collects vulnerability fixes from open-source projects and links them to CVE records and weakness types [15]. It covers many languages, including Python. SecurityEval offers Python examples for many weakness types [16]. Both are suitable sources for this project.

Datasets need care. Croft et al. studied data quality in vulnerability datasets. They found problems such as inaccurate labels and duplicate samples [17]. These problems can make a model look better or worse than it is. I will therefore check and clean the data, and I will report every cleaning step.

B.4 Explanations for developers

Many detectors output only a label or a probability. Without an explanation, a developer cannot check the result. Explanation methods already exist [13], [14]. In the papers I have reviewed so far, few Python vulnerability studies use them, and fewer still test whether the explanation points to the right code. This is where my project can add something.

B.5 Research gap

Three gaps follow from this review. First, I found little work that compares a learned model with Bandit on the same Python data. Second, hybrid methods that use Bandit output as input to a model are rarely examined. Third, explanations are seldom checked for usefulness. This project will address these gaps in a small and controlled way. The literature search will continue during the first two weeks, and I will update the gaps if I find closer work.

III. PILOT RESEARCH STUDY

A. Hypotheses

The aim is to find out whether a learned model can detect vulnerabilities in Python code better than Bandit, and whether combining the two helps. Two hypotheses will be tested.

H1: The machine learning model achieves a higher F1-score than Bandit on the same test set.

H2: The hybrid approach has a lower false-positive rate than Bandit alone and than the model alone, and its recall stays within five percentage points of the model.

The null hypotheses state that there is no difference. They will be tested at the 0.05 significance level.

B. Research Method

A small pilot will run in the first week. It will check that the plan is workable before the main experiments start. It has four steps.

1. Select about 200 Python snippets from the public datasets, with both vulnerable and safe code.
2. Run Bandit on every snippet and record each warning.
3. Extract simple features, such as the number of risky function calls, and train a basic random forest with five-fold cross-validation.
4. Compare the F1-scores and note any problems with labels or parsing.

The pilot is not a result of the dissertation. Its purpose is to test the data pipeline and the evaluation code.

C. Initial Results

No experimental results exist yet, because the pilot will run once the project starts. Literature gives some early signs. Chakraborty et al. show that models score lower on realistic data than on benchmarks [5]. I therefore expect modest F1-scores. Static tools in general raise warnings on harmless code [4]. I expect Bandit to find most of the simple patterns it has rules for, but with lower precision. These expectations are the basis for H1 and H2. They are not findings.

Two public datasets have been identified, CVEfixes and SecurityEval. Their licence terms will be checked in the first week.

IV. OUTLINE OF PROPOSAL

A. Aims of the Investigation

The aim is to design, build and evaluate an explainable machine learning tool that detects four types of security vulnerability in Python code, and to compare it with Bandit. The expected outputs are a cleaned dataset, a trained model, an offline application and a dissertation of 15,000 to 20,000 words.

B. Research Objectives

The project has nine objectives.

1. Review the literature on vulnerability detection, static analysis and explainable models.
2. Collect, clean and label a Python dataset for the four weakness classes.
3. Run Bandit on the dataset and record baseline results.
4. Build and tune machine learning models using code features.
5. Build and test the hybrid approach.
6. Add SHAP explanations and plain-English messages.
7. Develop the offline application.
8. Test the application and evaluate the results against the hypotheses.
9. Write the dissertation, keep the development evidence and prepare the disc.

The investigation will answer four questions.

RQ1: Which code features best predict vulnerable code?

RQ2: How do precision, recall and F1 of the model compare with Bandit?

RQ3: Does the hybrid approach lower the false-positive rate without losing recall?

RQ4: Do the explanations point to the code that makes a snippet risky?

C. Methodologies

Research approach. The study is positivist and quantitative [18]. It is an experiment that tests hypotheses with measured results. The product will be built in short iterations of one week. Each iteration ends with a working increment and a log entry.

Data. Python functions will be taken from CVEfixes and SecurityEval. The code before a fix will be labelled vulnerable. The fixed version will be labelled safe. Extra safe functions from the same projects will be added, so that the class balance is closer to real use. Duplicates will be removed. The train and test sets will be split by project, so that code from one project never appears in both. Fields that hold developer names or e-mail addresses will not be used.

Features and models. Python's ast module will extract features. Examples are counts of calls to eval, exec, os.system and pickle.loads, string joining inside database calls, and string values that look like passwords. Logistic regression, random forest and gradient boosting will be trained with scikit-learn [19]. Settings will be tuned with grid search inside cross-validation. If time allows, CodeBERT vectors [11] will be tested as extra features.

Baseline. Bandit warnings will be mapped to the four weakness classes. For example, B608 maps to SQL injection, B602 and B605 to command injection, B105 to B107 to hard-coded credentials and B301 to deserialisation. A function counts as flagged if it has at least one mapped warning.

Hybrid. Two designs will be tried. In the first, Bandit output becomes extra input features for the model. In the second, a warning is kept only when both the model and Bandit agree.

Controls. To limit overfitting, the test set will stay untouched until the final run. Tuning will use only the training data. The random seed will be fixed and recorded, so another person can repeat the experiments.

Evaluation. The study will report precision, recall, F1-score, false-positive rate and a confusion matrix for each method. McNemar's test on paired predictions will check whether differences are real [20]. For RQ4, I will take 50 correctly flagged snippets and compare the top SHAP features with the risky line. A written rubric will guide this check. No participants are needed.

Application. The tool will have a Streamlit interface and a command-line mode. Fig. 1 shows the planned structure. Results will be stored in SQLite. A scans table will hold the file name, time and model version. A findings table will hold the line number, weakness type, risk score, Bandit flag and explanation text.

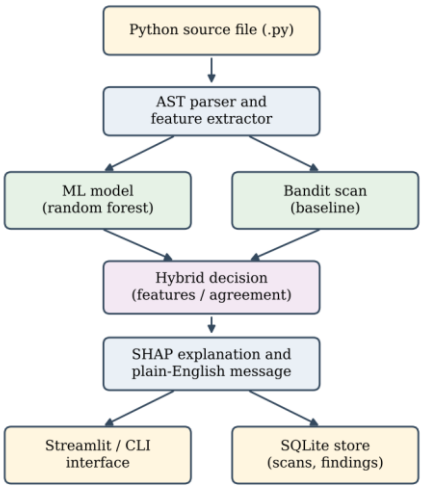


Fig. 1. Planned system architecture.

Testing and evidence. Unit tests will use pytest. The finished package will be run on a clean machine with no internet. Evidence will include Git history, a weekly log, notebook outputs, test reports and notes from supervisor meetings.

D. Ethical Considerations

There will be no human participants and no personal data. The datasets are public and will be used under their licences, which I will record. The project will not produce exploit code or attack tools. The tool only detects and explains. All testing will use code files and never live systems, which keeps the work within the law, including the Computer Misuse Act 1990. Vulnerable samples will stay inside the project and will not be shared beyond what the dataset licences allow. I will complete the university ethics checklist and confirm it with my supervisor before the experiments begin.

E. Plan and Timetable

The project starts on 19 October 2026 and ends with submission on 29 December 2026. This lasts about ten

weeks. Fig. 2 shows the plan. Writing runs alongside development, so the report is not left to the end. The last two weeks include the holiday period, so I have kept the final tasks small and left 28 December free as a buffer.

There are five milestones. The dataset will be ready by 6 November. The Bandit baseline will be done by 9 November. The models will be finished by 27 November. The application will work by 11 December. Testing will be completed by 17 December. I will meet my supervisor and keep notes of each meeting.

Three risks could delay the work. The datasets may be harder to clean than planned, so I will start with a small set and add more only if time allows. Training may be slow on a laptop, so I will use simple models first. The holiday period may reduce supervisor's access, so I will agree to meeting dates early.

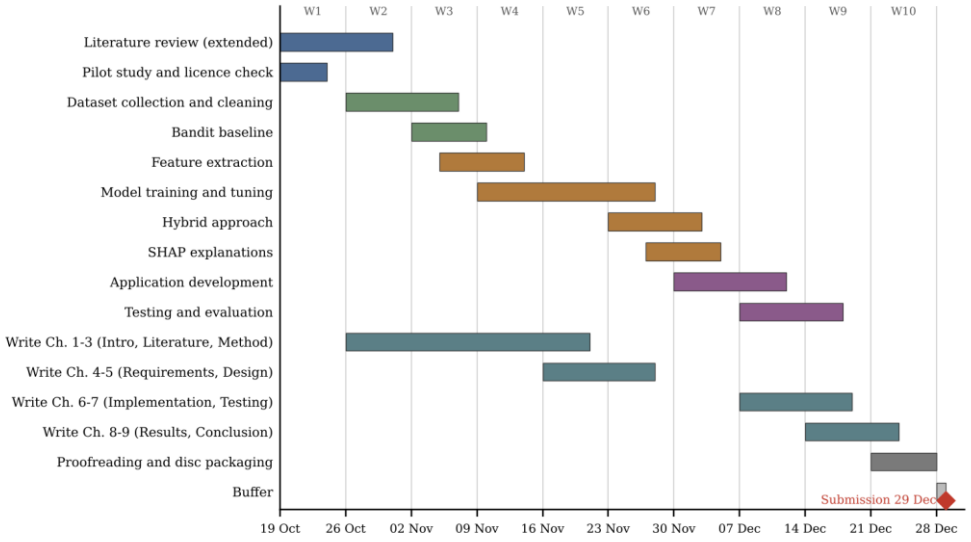


Fig. 2. Project timetable, 19 October to 29 December 2026.

F. Limitations and Scope

The project has clear limits.

- It covers Python only and four weakness classes.
- It works at function level, so it cannot follow data across files.
- Public datasets may still contain wrong labels after cleaning.
- Results may not be held for other languages or for all real projects.
- Bandit is the only baseline. Other tools are out of scope.
- Studies with developers as users are out of scope.
- Deep learning is optional, because the time is short.

The tool is a research prototype. It is not a replacement for code review or professional scanners.

G. Personal Development Requirements

I need to strengthen several skills. These are model evaluation and statistics, explainable machine learning, parsing Python code with the ast module, and writing a long report. I will use the university library, online courses and supervisor feedback. I will also practice Git and automated testing, because they provide the development evidence.

H. Resource Requirements

The project needs one laptop or PC with at least 8 GB of RAM. All software is free. It includes Python 3, Jupyter, scikit-learn, pandas, SHAP, Bandit, Streamlit, SQLite, pytest, Git and Matplotlib. If CodeBERT is used, the model files will be downloaded once and stored on the disc, so the tool still runs offline. The data sources are CVEfixes and SecurityEval. I will plan tasks with a free Gantt or Kanban tool. Microsoft Word will be used for writing. A recordable DVD or a CD drive will carry the final hand-over, which will hold the dissertation, source code, sample data, trained model, evidence and an installation guide.

REFERENCES

[1] OWASP Foundation, "OWASP Top 10:2021." [Online]. Available: https://owasp.org/Top10/

[2] M. Alfadel, D. E. Costa, and E. Shihab, "Empirical analysis of security vulnerabilities in Python packages," in *Proc. IEEE Int. Conf. Softw. Anal., Evol. Reengineering (SANER)*, 2021, pp. 446–457.

[3] PyCQA, "Bandit: A tool designed to find common security issues in Python code." [Online]. Available: https://github.com/PyCQA/bandit

[4] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why don't software developers use static analysis tools to find bugs?" in *Proc. 35th Int. Conf. Softw. Eng. (ICSE)*, San Francisco, CA, USA, 2013, pp. 672–681.

[5] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet?" *IEEE Trans. Softw. Eng.*, vol. 48, no. 9, pp. 3280–3296, Sep. 2022.

[6] N. Ayewah, D. Hovemeyer, J. D. Morgenthaler, J. Penix, and W. Pugh, "Using static analysis to find bugs," *IEEE Softw.*, vol. 25, no. 5, pp. 22–29, Sep./Oct. 2008.

[7] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2022, pp. 754–768.

[8] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[9] Z. Li *et al.*, "VulDeePecker: A deep learning-based system for vulnerability detection," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, San Diego, CA, USA, 2018.

[10] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019.

[11] Z. Feng *et al.*, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings Assoc. Comput. Linguistics: EMNLP 2020*, 2020, pp. 1536–1547.

[12] H. Hanif, M. H. N. M. Nasir, M. F. Ab Razak, A. Firdaus, and N. B. Anuar, "The rise of software vulnerability: Taxonomy of software

vulnerabilities detection and machine learning approaches," *J. Netw. Comput. Appl.*, vol. 179, Art. no. 103009, 2021.

[13] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Adv. Neural Inf. Process. Syst. (NIPS)*, vol. 30, 2017, pp. 4765–4774.

[14] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 1135–1144.

[15] G. Bhandari, A. Naseer, and L. Moonen, "CVEfixes: Automated collection of vulnerabilities and their fixes from open-source software," in *Proc. 17th Int. Conf. Predictive Models Data Anal. Softw. Eng. (PROMISE)*, 2021, pp. 30–39.

[16] M. L. Siddiq and J. C. S. Santos, "SecurityEval dataset: Mining vulnerability examples to evaluate machine learning-based code generation techniques," in *Proc. 1st Int. Workshop Mining Softw. Repositories Appl. Privacy Secur. (MSR4P&S)*, 2022, pp. 29–33.

[17] R. Croft, M. A. Babar, and M. M. Kholoosi, "Data quality for software vulnerability datasets," in *Proc. 45th Int. Conf. Softw. Eng. (ICSE)*, 2023, pp. 121–133.

[18] B. J. Oates, M. Griffiths, and R. McLean, *Researching Information Systems and Computing*, 2nd ed. Thousand Oaks, CA, USA: SAGE, 2022.

[19] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[20] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Comput.*, vol. 10, no. 7, pp. 1895–1923, 1998.